

COSC 4370 Homework 3

Raphael Tinoco 1668116

October 17, 2023

1 Assignment Problem

This assignment had two main tasks. The first was to implement 3D viewing. The second was implementing the Phong shading model.

2 Methods Used

Methods used for implementing 3D viewing:

Within the camera.h header file, I was tasked with updating the GetViewMatrix() function to have it return the View Matrix. This was done by utilizing the glm function lookAt(), which takes in three vectors as parameters. These vectors are then stacked on top of each other as rows within a 4x4 matrix (bottom row being 0 0 0 1). This matrix is then multiplied by a translation matrix with its final column being the camera's negated position vector. This multiplication produces the view matrix that is used to transform all world coordinates into the camera coordinate system. This view matrix is returned by the GetViewMatrix() function.

Within main.cpp, I was tasked with simply setting up the perspective projection matrix. This is necessary as the code allows for the user to move around freely in the world, and as such, the graphics transformation pipeline would require a perspective projection over an orthographic one. I did this by utilizing the glm function perspective() which takes in 4 parameters: the fov angle (in radians), the aspect ratio of the screen, the near clipping plane, and the far clipping plane. We can use the created camera object's Zoom member variable for the fov parameter and examine the window dimensions specified in main.cpp for the aspect ratio. Then, we can arbitrarily choose clipping plane values for the final two parameters. Doing so will have the perspective() function return the perspective matrix that we can now refer back to.

Methods used for implementing the Phong shading model:

The Phong shading model is divided into the phong.vs and phong.frag files. In phong.vs, I use the following methods: vec3(), vec4(), inverse(), transpose(), and mat3().

The first two methods are similar in that they convert a given vector or coordinates into its type (i.e. vec3 can extract the first three coordinates from a 4D vector to create a 3D vector, or you can have vec4 take in the coordinates of a 3D vector and one additional coordinate, usually 1, to create a 4D vector).

Inverse() simply returns the inverse of the matrix passed as parameter. Likewise, transpose() transposes a given matrix. Finally, mat3() extracts a 3x3 matrix from its parameter matrix.

In phong.frag, I use the following methods: normalize(), dot(), max(), reflect(), pow(), as well as the already established vec3(), vec4().

Normalize() takes in a vector and normalizes it so that calculations can be simplified. Dot() calculates and returns the dot product between two vectors and is very useful for the Blinn Phong Reflection model. Reflect() takes in two parameters, a surface norm and light direction. These are used to calculate and return a reflected vector that is useful in specular term calculations. Max() and pow() are basic mathematical functions.

3 Details of Implementation

The implementation of 3D viewing was mostly given to us as code, and we simply had to add in 2 lines of code. One allowed for the view matrix to be returned through a function, and the other specified the projection matrix.

Phong.vs

All that remained following such additions was to implement the Phong shading model. We begin with the phong.vs file in which we are given as input a vertex's local position and its local normal. First, we translate the position to the 2D screen space following the transformational pipeline; convert to a homogeneous coordinate, and apply the modeling transformation. At this point, we get a 3D vector and output it as FragPos to be used by the fragment shader.

Next, we reconvert it back into a 4D vector to then have it undergo the camera transformation then the projection transformation (both specified earlier). We assign this final position to gl_Position so that the rest of the rendering pipeline can occur for the vertex. Also, we want to calculate the transformed Normal vector (in world space) from the given local normal. This is done by taking the inverse of the modeling transformation, transposing it, setting it as a 3x3 matrix, and then having this matrix multiply by the normal. We store this to Normal to be output for use by the fragment shader.

Phong.frag

The file takes in as input two 3D vectors: FragPos and Normal, both calculated and output from the phong.vs file mentioned previously. Also, we have 4 uniform 3D vectors: lightPos, viewPos, lightColor, objectColor. The model consists of three main parts: ambient lighting, diffuse lighting, and specular lighting.

Ambient lighting:

We begin by specifying an arbitrary ambientCoefficient, and create a 3D vector that is the product of this coefficient and lightColor. We store this resulting 3D vector as ambient. With this, we have calculated the part of ambient lighting within the model.

Diffuse lighting:

First, we normalize the Normal vector to allow for simplistic calculations and store it as `norm`. We then calculate the light's direction by taking the difference between `lightPos` and `FragPos` and normalize for the same reason, storing it as `lightDir`. Then, we take the dot product of `norm` and `lightDir` and the max of this value and 0 to calculate the amount of energy being received, storing it as `energyReceived`. This value will range from 0 (when `norm` and `lightDir` are exactly perpendicular) and 1 (when the angle between them is 0). Finally, we multiply `energyReceived` with `lightColor` and store the resulting 3D vector as `diffuse`. Now, all that remains is to calculate the specular vector.

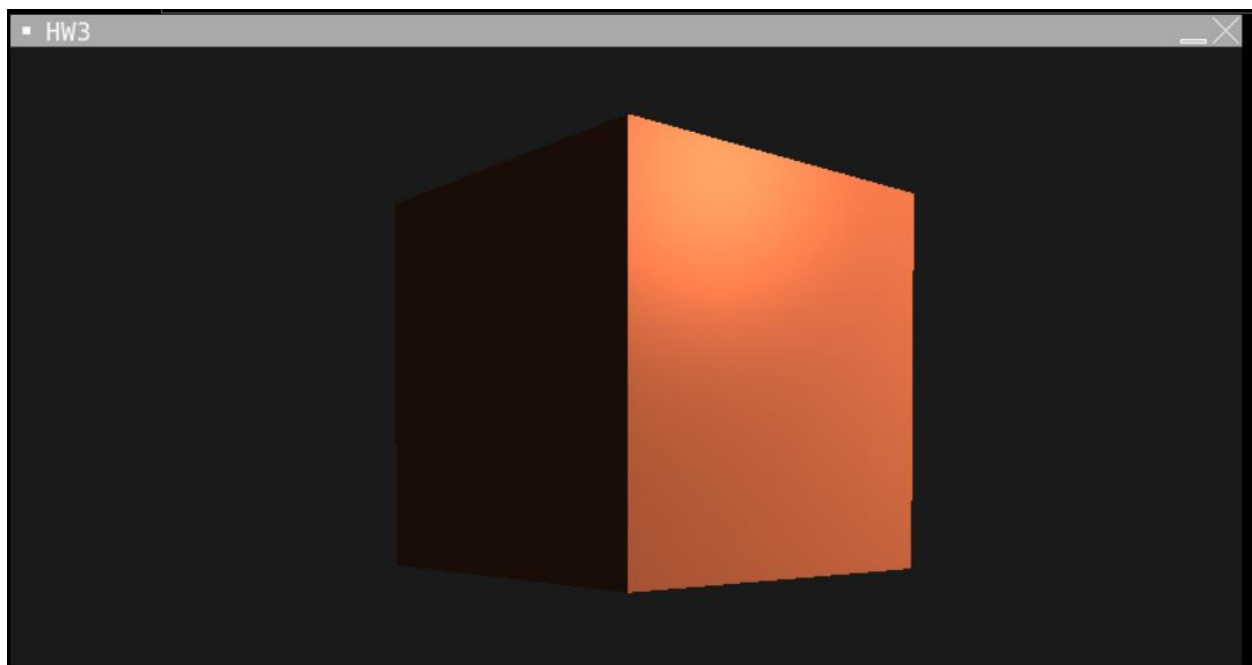
Specular lighting:

Similar to ambient lighting, we arbitrarily select a `specularCoefficient`. Next, we want the view direction vector between `LightPos` and `FragPos` as we want to reflect the angle across `norm`. So, we calculate `viewDir` by the normalized difference of `viewPos` and `FragPos`. Then, we reflect across the norm by using `reflect(-lightDir, norm)` and store it as `reflectDir`. With this reflected vector, we can now calculate the amount of energy received in specular lighting by taking the dot product of `viewDir` and `reflectDir`, and getting the max of this and 0. At this point, we would raise the value to the power of a shininess variable `p`, but we chose the value to arbitrarily be 32. For reference however, the larger the value of `p` is, the more narrower the lighting area. We store this final value as `spec`. To generate the 3D vector `specular`, we multiply the `specularCoefficient` with `spec` and `lightColor`. We now have all three components.

The resulting 3D vector aptly named `result` is calculated by the product of `objectColor` with the sum of all three above components. Then, we assign the 4D color vector `vec4(result, 1.0f)`.

Finally, the Blinn Phong Shading model is done.

4 Results



As seen in this resulting image, the cube has different color values thanks to the Blinn Phong Shading model. In places where light is not reaching, the color is determined solely by the ambient lighting value. Else, if given light, the intensity depends on the viewing angle of the shape.

5 Other Materials

Links used to work on project:

1. <https://learnopengl.com/Getting-started/Camera>
2. <https://learnopengl.com/Lighting/Basic-Lighting>